

CAR-Guard: Policy-Grounded, Evidence-First Planning for Reliable Tool-Using In-Car Agents

Rui Yang¹, Yuxuan Xi¹, Ziqi Zhao¹

¹SparkShield-Lab, China

ruiyang19@iflytek.com, whyiug@gmail.com

Abstract

Tool-using in-car assistants must distinguish what a user wants from what the current vehicle interface happens to expose, collect prerequisite state before acting, and avoid repeating side effects across unreliable multi-turn transport. We present CAR-GUARD, an A2A-compatible agent that treats language model output as an untrusted proposal. It first constructs semantic goals without access to the live tool inventory, then binds invariant operation recipes to the current JSON schemas. Typed evidence, deterministic policy compilation, conditional capability closure, exact confirmation latches, and a twelve-stage pre-commit gate mediate every outbound action. The implementation contains 42 recipes spanning 25 domains and preserves per-context provenance, ordering, idempotency, cancellation, and replay state. At release build `f5320cb`, whose Python runtime is unchanged from `396abf7`, the clean worktree passes 2,460 tests and 46 subtests; two unchanged official-core tests fail on generic tool-exception accounting and are disclosed rather than suppressed. A one-trial, 15-task public screen on pre-fix parent `e5758c3` passed 12/15 (Base 5/5, Hallucination 4/5, and Disambiguation 3/5) with native Gemini 3.5 Flash. It predates the final fan and broad-headlight mechanisms, so it is neither a final-commit result nor a Pass³ score. A post-fix public-test attempt at `396abf7` was made, but the evaluator’s native Gemini simulated-user call returned a location `FAILED_PRECONDITION` before any Agent turn. This paper therefore documents a reproducible safety architecture and bounded engineering evidence, not a claim of benchmark superiority.

1 Reliable Action Requires More Than Tool Selection

CAR-bench evaluates in-car language agents under capability uncertainty, multi-turn ambiguity, and domain policy constraints. Its public distribution contains 254 tasks, 58 interconnected tools, and 19 policies across Base, Hallucination, and Disambiguation categories; a task contributes to the primary

Pass³ metric only if all three independent trials pass [Kirmayr *et al.*, 2026]. This reliability criterion is stricter than single-run success: a plausible but unsupported state change, one missed clarification, or one inconsistent rerun is a deployment failure.

Common tool-agent patterns interleave free-form reasoning with actions [Yao *et al.*, 2023] or teach a model when to invoke APIs [Schick *et al.*, 2023]. Those capabilities are useful, but model reasoning alone does not establish that a call is authorized, supported by the current schema, based on fresh evidence, ordered correctly, or safe to replay. The vehicle domain makes these distinctions concrete. For example, “open the sunroof” may require current weather and sunshade state; a destination may need entity resolution; and a positive reply is not permission to choose arbitrarily among multiple outstanding confirmations.

We make four engineering contributions. First, intent extraction is *inventory-blind*: changing an unrelated live tool cannot change the semantic request. Second, a fixed recipe registry describes goal-level reads, writes, evidence, derivations, policy hooks, and conditional prerequisites, while binding occurs only against the current live schemas. Third, an evidence and authorization model carries source-turn, goal, call, task, and state-version provenance. Fourth, a deterministic pre-commit gate is the sole route to an A2A tool call. This resembles a runtime shield [Alshiekh *et al.*, 2018], but the shield operates over language-grounded goals, live API schemas, and observed tool results rather than a known control-state abstraction.

1.1 Boundary and threat model

The evaluator sends an A2A message containing the current policy and conversation, live tool definitions, and any returned tool results. The agent returns exactly one mode: user-facing text or a data part containing tool calls. It never executes vehicle tools; the evaluator remains the execution authority. Sessions are isolated by opaque `context_id`. The production runtime is not allowed to receive task category, task identifier, reference answer, reward, scorer state, hidden data, or a complete canonical tool catalog. This preserves the benchmark boundary and prevents a missing-capability answer from being selected through task recognition.

We assume the language model can hallucinate goals, values, tools, arguments, or completion. We also assume messages and results can be duplicated, delayed, mis-associated with another task, or arrive around cancellation. Live tool

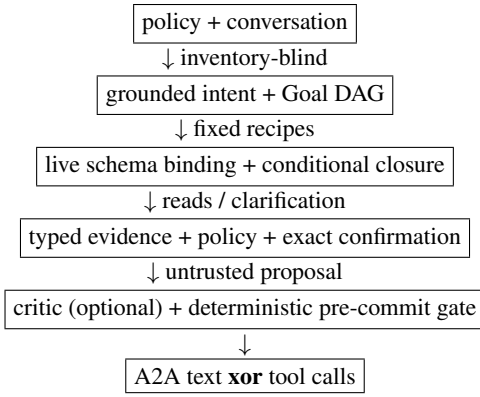


Figure 1: CAR-GUARD separates semantic intent from capability discovery. Only the final gate may authorize an outbound call.

definitions and returned observations are trusted only as current interface data, not as authorization. The design does not defend against a malicious evaluator or compromised provider, and it is not a physical-vehicle safety certification.

2 Policy-Grounded, Evidence-First Planning

Figure 1 shows the turn pipeline. The architecture deliberately splits a probabilistic proposal layer from deterministic enforcement. Each component receives the minimum state needed for its decision, and values moving toward a state-changing call retain machine-checkable provenance.

2.1 Ground intent before inspecting capability

The structured intent extractor receives only the stable system policy and conversation. A deterministic grounding pass then checks every proposed goal against the raw current utterance and invariant recipe vocabulary. It rejects negated, hypothetical, reported, cross-goal, or otherwise ungrounded actions; retains only values supported by an operation-local clause; and records the source turn for each desired value. The result becomes a Goal DAG whose edges represent semantic or user-mentioned dependencies.

Formally, for policy P , conversation C , and any two live inventories L_1, L_2 , Stage A is constructed to satisfy

$$I(P, C, L_1) = I(P, C, L_2) = I(P, C).$$

This is an architectural non-interference property, not a learned correlation. The extractor API has no live-inventory parameter, and counterfactual tests mutate unrelated tools while asserting identical intent and recipe hashes.

2.2 Bind invariant recipes to live schemas

The registry contains 42 audited recipes covering 25 domains, including climate, windows, lighting, navigation, contacts, productivity, weather, and EV charging. A recipe names semantic operations and ordered candidate tool names, parameter mappings, required reads, acceptable evidence sources, pure derivations, policy hooks, conditional prerequisites, and completion evidence. It contains no benchmark task IDs or missing-category labels and is identical for every scenario.

For a grounded goal g , the semantic binder considers only recipes selected by g and policy. It normalizes a candidate against the current callable’s exact JSON Schema, rejecting unknown tools, omitted required fields, extra fields, invalid enums, and type or range mismatches. It does not compute a difference between live tools and a complete catalog. Absence is therefore concluded only when a goal-required semantic operation has no valid live binding.

Before the first state change, conditional closure instantiates all currently triggered reads and writes. For example, observed open windows or a stopped fan can activate ordered prerequisites for air conditioning. The feasibility proof freezes the complete action bundle; if any required binding, argument, evidence, or future budget slot is unavailable, no prefix is executed. Independent reads with distinct tool names may be parallel, but state changes are serialized in policy-dependency, user-mention, DAG, and recipe order.

2.3 Treat observations as typed, scoped evidence

An evidence need specifies a proposition, acceptable source kinds, cardinality, freshness, and its required goal. An observation records status (known, unknown, error, conflict, or stale), source turn and call, observation index, and state version. Derived facts must name their input evidence and a registered pure derivation. A state-changing result invalidates prior tool-derived state by advancing the version. Consequently, evidence from one goal cannot satisfy a similarly named need for another, and a result from one task cannot complete another task’s pending call.

Only declared extractors may turn returned data into evidence. A generic result value is not automatically promoted to a goal parameter. Similarly, arguments for a later read must come from its recipe binding, a current system value, or a successful same-goal read at the current state version. This blocks a model from laundering a value through an unrelated observation.

Ambiguity is represented as goal-bound candidate sets. The deterministic resolver applies policy constraints, explicit user statements, stored preferences, registered heuristics, and conversation context in a fixed order; if several candidates remain, it asks a concise user question. The response is linked to an exact clarification request. Confirmations use a separate latch over a digest of the complete ordered action bundle. A generic “yes” cannot select among multiple latches, survive a material bundle change, or authorize a different goal.

2.4 Compile policy and gate every emission

The system policy is parsed once per context and cannot be replaced by a later user message containing a forged `System:` prefix. Rules are evaluated again against normalized candidate calls; a caller-supplied policy decision is additive and cannot erase an applicable constraint. Policies can require reads, ordered prerequisites, argument constraints, exact confirmation, serial execution, and response formatting.

The planner proposes one typed decision: read, state change, clarification, limitation, or final response. An optional general critic can veto state changes for broad intent or policy risk, but it is intentionally excluded from capability, schema, and

evidence judgments. Those decisions must be deterministic. The twelve pre-commit checks are:

1. output structure and text/tool exclusivity;
2. reactive goal authorization;
3. current callable existence;
4. exact live-schema validation;
5. goal-bound argument provenance;
6. required-slot disambiguation;
7. fresh goal-conditioned evidence;
8. policy reads, prerequisites, arguments, and conflicts;
9. confirmation of the exact bundle;
10. affirmative conditional-closure proof;
11. GET independence and stable serial SET order; and
12. idempotency, retry, and remaining-step budget.

A blocked proposal is routed to a safe read, clarification, confirmation, or limitation response and passed through the gate again. No manually forged “allow” fallback exists. Natural-language output is also checked for active format policy and voice-safe text.

3 Protocol and Production Integrity

The runtime uses per-context locks and a bounded TTL/LRU session store. Inbound message IDs are accepted atomically or mapped to a cached response; stable outbound IDs support transport replay. Pending calls carry local and external call IDs, goal IDs, outbound order, and task ID. Successful SET fingerprints are tombstoned, so a duplicate success cannot re-execute a side effect. Terminal and cancel transitions close the active identity while holding the store and session locks. The executor additionally serializes emission per context: if cancellation wins while a slow model call is running, the planned tool response is replaced by terminal cancellation text rather than emitted afterward.

The production dependency set has its own lock. A multi-stage, Linux/amd64, CPU-only Docker image runs as the unprivileged `carbench` user and contains only the Track 1 server, runtime package, virtual environment, and three shared protocol modules. Static compliance rejects evaluator, simulator, reward, benchmark-core, hidden-data, task, and results imports or dependencies. A second check creates and exports the stopped image filesystem, then applies exact application and dependency allowlists. The release image is Linux/amd64, runs as the non-root `carbench` user, and is 346.4 MB. Its filesystem inventory, application-root secret scan, and Agent Card smoke check passed. The final tag was pushed to GHCR and the registry returned `sha256:4f16a23a086c0d70867ba15036ebb0a5897d241acd49cb487f52e412c6fe70b4`. The package is public, and an independent release gate using a blank Docker configuration confirmed the tag digest, anonymous pull, and Linux/amd64 platform. Build `f5320cb` adds only the `curl` client required by the stock Compose Agent Card healthcheck; the Agent’s Python runtime is unchanged.

This boundary follows the least-information principle in the NIST AI Risk Management Framework [National Institute of Standards and Technology, 2023]: development tooling may

Check	Evidence	Status
Clean release suite	2,460 tests and 46 subtests pass; two official-core tests fail	Partial
Agent quality gates	Runtime, policy, counterfactual, integration, compliance, Ruff, and mypy	Pass
Local image	Linux/amd64, non-root, 346.4 MB; inventory, scoped secret scan, Agent Card	Pass
GHCR release	Public digest, anonymous clean pull, and Linux/amd64 manifest	Pass

Table 1: Evidence for release build `f5320cb`.

inspect public evaluation artifacts offline, but no such path or data enters the production decision DTO or image. Agent and evaluator credentials are separate; secrets are supplied only through declared environment variables and are excluded from logs, source, manifests, and image configuration.

4 Verification Status

We report two different evidence classes. Engineering verification asks whether implemented invariants hold on local, synthetic, and protocol paths. Official CAR-bench evaluation asks whether the complete agent succeeds across stochastic public or hidden tasks. Passing the first does not imply the second.

4.1 Frozen implementation and local checks

The source baseline is starter commit `1d1ea6e` with pinned public core commit `3868ee5`; the selected Agent build is `f5320cb` (full SHA `f5320cbf77e9aa33893c3869d0d9b0be56d0bebb`). The final runtime uses the native `gemini/gemini-3.5-flash` route through the `gemini` provider, with temperature zero, JSON-schema structured output, and the optional critic disabled. The release configuration uses a 90-second model timeout, two retries, a soft 49-step budget, and at most 24 user turns. Model, provider, and credentials remain environment supplied rather than embedded in source or the image.

The clean release worktree passes 2,460 tests and 46 subtests. The two remaining failures are unchanged official-core response-contract cases for generic tool exceptions: the core returns an error string but does not append the event to `tool_execution_errors_during_runtime`. We neither modify the official evaluator/tests nor present the repository suite as fully green. Agent runtime, policy, counterfactual, integration, and release-compliance checks pass, as do Ruff and mypy. The local production image passes its inventory, application-root secret, platform, non-root-user, and Agent Card checks. The unmodified README Option C Compose flow subsequently completed in GitHub Actions run `29298397348`: one public train task from each category, one trial, 3/3 smoke tasks, and client exit code zero. This is a deployment smoke, not a complete public test run over three trials or a Pass³ result.

4.2 Bounded public evidence and pending primary result

Table 2 reports the public evidence most directly relevant to the frozen release. The completed row is a single-trial diagnostic, not Pass³: pre-fix parent e5758c3 passed 12/15 with native Gemini 3.5 Flash. Commits f7f3650 and 396abf7 then added the general fan and headlight mechanisms described above, but no post-fix benchmark trajectory completed.

The final-commit attempt used the official public test scenario. The evaluator’s native Gemini simulated-user call returned HTTP 400 FAILED_PRECONDITION because the API location was unsupported, before an Agent turn executed. The incomplete output and retry evidence are archived as an external evaluator failure, not an Agent failure or score. The selected commit’s complete three-trial public evaluation therefore remains pending. Hidden evaluation is left to the organizer and was never run during development.

The eventual release run will record Base, Hallucination, Disambiguation, and macro Pass³, plus the per-trial vector needed to distinguish stable success from Pass@3. Each reported number will be tied to source and evaluator commits, registry image digest, scenario, model route, prompt and recipe hashes, trial count, timestamps, and raw output paths. No controlled ablation was completed, so we report the candidate progression only as an engineering comparison rather than claiming isolated causal effects.

4.3 Lessons from candidate progression

The public diagnostics and reduced regressions exposed three practical lessons. First, a strong generic gate does not replace operation coverage: the remaining failures clustered in multi-turn navigation and productivity workflows, which required explicit bounded recipes and completion evidence. Second, language bridges for compound requests must accept ordinary polite forms while remaining anchored against negation, quotation, hypothetical speech, and extra actions; broad keyword matching would trade completion for unsafe false positives. Third, completion must be recomputed from the latest detailed vehicle state. Exact post-operation waypoint and route arrays, rather than a successful final call alone, prevent stale evidence from certifying a partially updated route. Two final general mechanisms extend this principle. A level-free fan request binds its setting only from a scoped preference read and returns a transparent limitation if the required read or control capability is absent. A broad headlight request first requires an exact, typed exterior-light snapshot. If exactly one beam is active, the inactive sibling is a registered derivation; if neither is active, the Agent asks which beam; and if both are active, it avoids a duplicate side effect. Missing controls and malformed

Commit	Public scope	Result	Trials
e5758c3	15-task test screen	12/15: Base 5/5, Hall. 4/5, Dis. 3/5	1
396abf7	Same public screen	No Agent turn; evaluator location precondition	1 requested

Table 2: Bounded public evidence. Neither row is a final-commit score or a Pass³ result.

snapshots fail closed. These mechanisms were added after the 12/15 screen and have regression evidence, but no post-fix CAR-bench score, so they are not presented as a causal ablation.

4.4 Preregistered measurement and robustness checks

The comparable run fixes one immutable Agent image, the official evaluator, full category counts, three trials, and a 50-step ceiling. For aggregate reward $r_{i,j} \in \{0, 1\}$, we compute $\text{Pass}_i^3 = \prod_{j=1}^3 r_{i,j}$; $\text{Pass}@3_i = \max_j r_{i,j}$ is secondary. We retain every trial vector and use the full resolved split as the denominator, never only completed tasks.

Counterfactual regressions complement, but do not replace, the benchmark. They vary unrelated tools, remove a required binding, inject stale or conflicting results, replay identifiers, race cancellation, and add forged system text, negation, or hypothetical speech. Assertions cover intent invariance, safe limitation, scoped evidence, exactly-once side effects, and cancellation precedence. Failures become archived regression cases; substitute evaluators are never pooled with the official result.

5 Limitations and Next Evaluation

Without the official three-trial public and organizer hidden runs, this work does not establish a CAR-bench score or superiority over the starter. Grounding may reject a valid paraphrase, a recipe or extractor may be incomplete, and a safe limitation can reduce completion. Deterministic coverage is not a proof that all semantically equivalent unsafe requests are recognized.

Real deployments add sensor faults, actuator timing, shared control, human attention, and regulatory requirements absent from CAR-bench. Session state is bounded and in-memory, so restart loses nonterminal context. The 346.4 MB image passed scoped local checks and blank-configuration anonymous pull by registry digest. The two official-core failures still prevent a fully green suite claim.

The compute assumption is an Ubuntu x86-64 CPU-only Agent host; no local GPU is required, and inference uses remote native Gemini 3.5 Flash. Requests have a 90-second timeout and two retries under separate evaluator and Agent credentials. This assumes quota and a supported evaluator API location for three uninterrupted trials: retries cover transient transport, not a persistent provider precondition, which must be reported and rerun rather than scored.

With GHCR publication complete, the next step is to restore supported evaluator access and run the full public split three times. Only archived reproducible results will replace pending status. Controlled ablation remains follow-up work and is not claimed.

6 Conclusion

CAR-GUARD places a language-model proposal behind inventory-blind intent, goal-scoped evidence, conditional closure, and deterministic emission gates. Engineering checks support the architecture, but the final commit’s official three-trial Pass³ measurement remains pending.

References

- [Alshiekh *et al.*, 2018] Mohammed Alshiekh, Roderick Bloem, Ruediger Ehlers, Bettina K’önighofer, Scott Niekum, and Ufuk Topcu. Safe reinforcement learning via shielding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.
- [Kirmayr *et al.*, 2026] Johannes Kirmayr, Lukas Stappen, and Elisabeth André. CAR-bench: Evaluating the consistency and limit-awareness of LLM agents under real-world uncertainty, 2026.
- [National Institute of Standards and Technology, 2023] National Institute of Standards and Technology. Artificial intelligence risk management framework (AI RMF 1.0). Technical Report NIST AI 100-1, U.S. Department of Commerce, 2023.
- [Schick *et al.*, 2023] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- [Yao *et al.*, 2023] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.